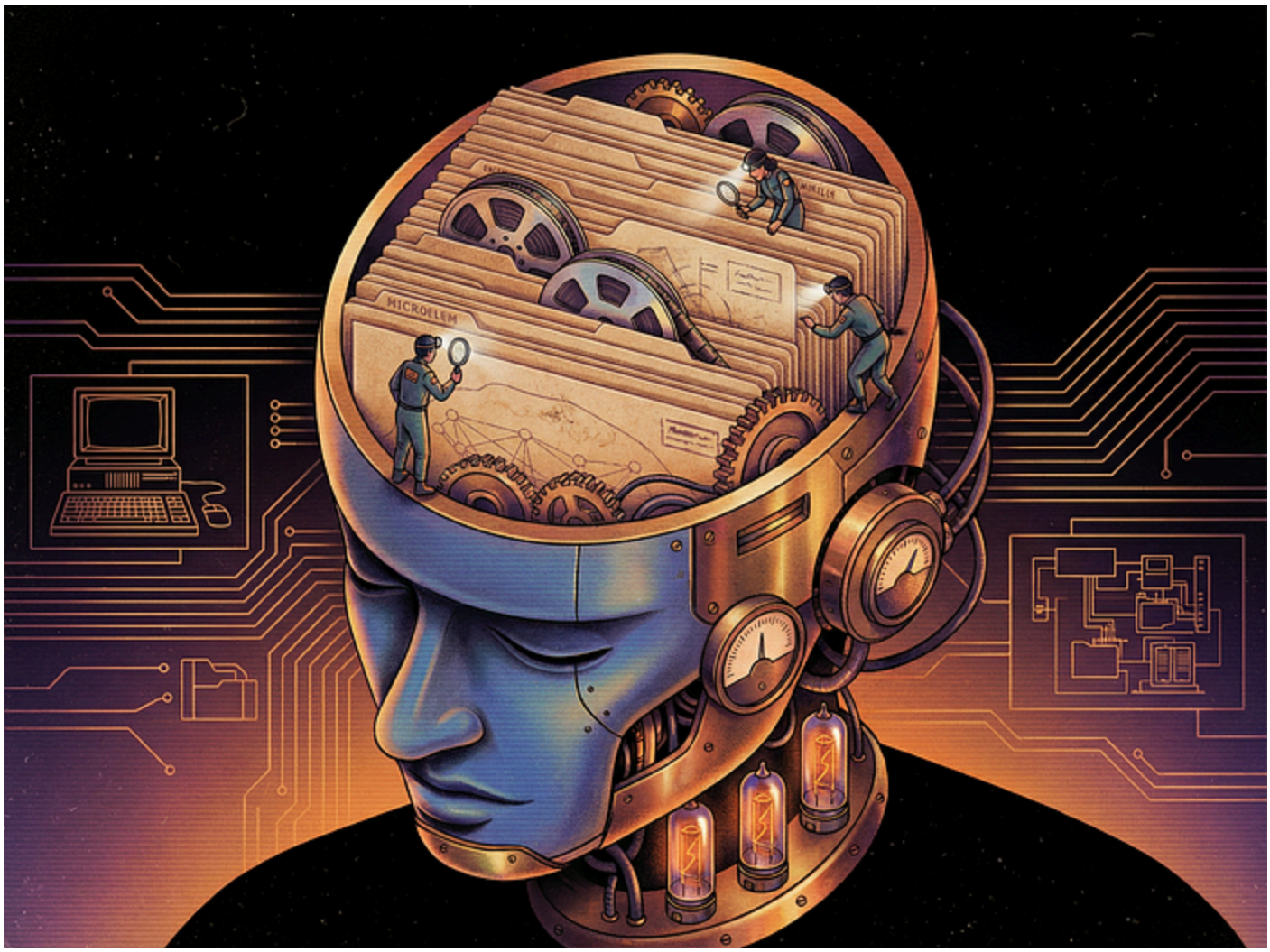
Milestone: 20GB Reached

We’ve reached 20GB of stored data — thank you for helping us grow!

Patreon Ko-fi Liberapay
Close
Freedium

- ko-fi
- librepay
- patreon

< Go to the original



OpenClaw Memory Systems That Don't Forget: QMD, Memo, Cognee, Obsidian

If your agent has ever randomly ignored a decision you know you told it... it's not random.

Agent Native Follow

~12 min read · February 20, 2026 (Updated: February 20, 2026) · Free: No

It's your memory system.

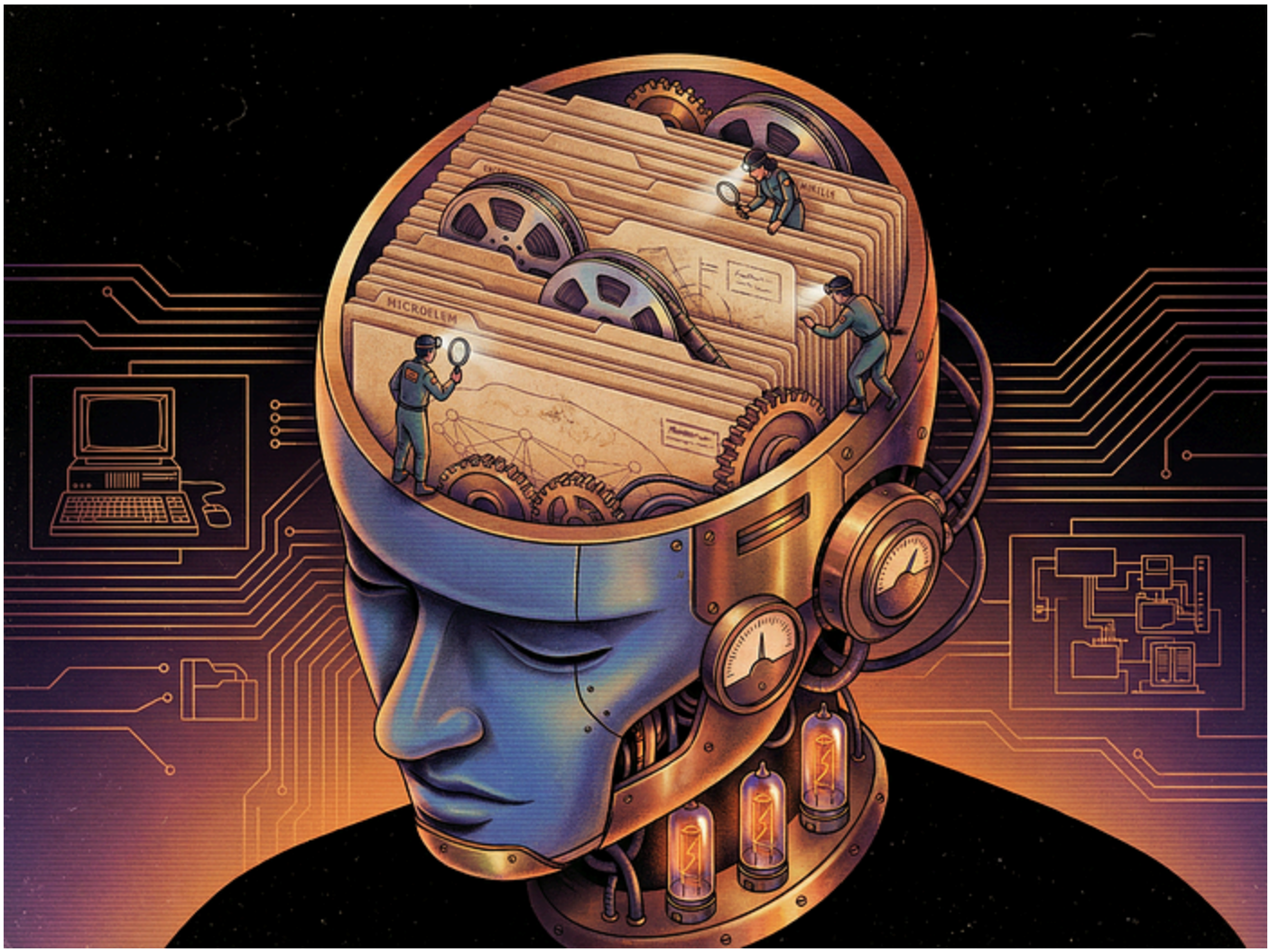
We were mid-migration with one hard rule: **after cutover, do not write to the old table**. We agreed on it explicitly and the next day the agent generated code that wrote to both old and new tables **just to be safe.**

If you're building multi-step workflows, e.g. cutovers, runbooks, incident response, multi-day projects, this failure mode is the default unless you engineer around it.

So here's what we're going to do:

- Fix OpenClaw memory at the harness level (flush checkpoints, working-set control, hybrid retrieval, session indexing).
- Define a **memory contract** so your agent is forced to search and forced to persist constraints.
- Then, when the harness stops being enough, look at our options to upgrade the substrate with **QMD**, **Mem0**, **Cognee**, and **Obsidian**.

By the end, you'll know exactly which memory failure you're dealing with, e.g. missed writes, missed retrieval, or compaction loss, and what to change first.



Memory is now more important than ever

I recently wrote about persistent memory for Claude Code

Persistent Memory for Claude Code: “Never Lose Context” Setup Guide

Before we start, it’s hard to ignore how much momentum long-term memory is getting in AI right now, especially from…

medium.com

and mentioned that long-term memory is the next big battleground for AI agents as they still struggle to **remember** across sessions, tools, and products.

We are more recently talking about multi-day horizon tasks, specialist agents and a **chief of staff** agent that keeps the plan.

A pipeline of agents passing context between them and amemory folder that starts to look like an actual project repo.

There is a constraint that doesn't care how smart your model is:

- Models are stateless between calls.
- Context windows are bounded.
- Anything not made durable will eventually fall out of view.

So **memory** becomes the difference between:

- an assistant that can help you execute a plan across sessions and
- a very confident autocomplete engine with amnesia.

That's why memory is suddenly everyone's problem.

But for OpenClaw, memory is just a plain Markdown in the agent workspace. How should you treat it?

Memory as a read/write/garbage-collection system

In practice, memory behaves like three separate subsystems:

1. **Write path:** what gets extracted and persisted, and when.
2. **Read path:** how the system retrieves and injects relevant state.
3. **Compaction/GC:** what gets summarized, pruned, or dropped to stay within token budgets.

If you design each path deliberately, memory is debuggable.

OpenClaw's default posture is basically:

- writes are discretionary
- reads are optional
- compaction is aggressive when you hit limits

This is actually painful for real systems.

The simplest way to think about it

Treat agent memory like a production service with three SLOs:

- **durability:** important facts and decisions survive session boundaries and long conversations
- **retrievability:** relevant facts get surfaced when they're needed
- **stability under compaction:** long sessions don't silently erase constraints

Fixing memory like an architect

1) Add a checkpoint before compaction (memory flush)

A practical OpenClaw pattern is enabling a flush step that runs before compaction, with a prompt that captures operationally useful state.

Here's an example config structure:

```
{
  "compaction": {
    "memoryFlush": {
      "enabled": true,
      "softThresholdTokens": 32000,
      "prompt": "Write a durable session note to memory/YYYY-MM-DD.md. Capture: decisions, constraints, open questi
      "systemPrompt": "Be terse. Prefer bullet points. Do not rewrite the conversation."
    }
  }
}
```

Why I like this framing:

- "decisions and constraints" are what break systems when lost
- "owners" matters if you're coordinating agents or tasks
- "open questions" prevents the agent from re-asking the same thing later

Trade-off is that you pay extra tokens and latency, and if your flush prompt is sloppy, you will store junk and retrieval will get noisy.

Do not ship long-running agents without a checkpoint strategy.

2) Control the working set with TTL pruning (context pruning)

Keep the recent window usable without bloating every turn.

TTL-based pruning is a blunt instrument, and blunt instruments are fine when you're getting punched in the face by token growth.

Example:

```
{
  "contextPruning": {
    "mode": "cache-ttl",
    "ttl": "4h",
    "keepLastAssistants": 4
  }
}
```

TTL pruning helps with:

- keeping the **recent** part of the conversation coherent
- reducing token spend

But it does not help with retaining anything beyond the TTL window unless you flush it

So treat it like a cache policy that protects your interactive loop, not your source of truth.

3) Make retrieval work for how engineers actually query (hybrid search)

You should handle both fuzzy queries ("the rate limit decision") and exact queries ("HTTP 429" / "PaymentService" / "JIRA-1842").

In agent systems, retrieval quality is often the real bottleneck, and if retrieval is weak, you can have perfect memory files and still act forgetful.

Hybrid search is one of those **unsexy but correct** engineering moves: combine dense semantic retrieval with lexical retrieval so you don't miss exact tokens.

OpenClaw-style config shape:

```
{
  "memorySearch": {
    "enabled": true,
    "sources": ["memory", "sessions"],
    "query": {
      "hybrid": {
        "enabled": true,
        "vectorWeight": 0.6,
        "textWeight": 0.4
      }
    }
  }
}
```

Two notes from experience:

- If your corpus contains lots of identifiers (codebases, logs, tickets), give lexical matching more weight than you think.
- If your corpus is mostly narrative notes, semantic weight can dominate.

4) Make the past queryable (session indexing)

Let the agent retrieve **what we decided last week** without you manually reconstructing the timeline.

Indexing session transcripts alongside memory files turns your conversation history into a searchable artifact.

This is important because agents often make decisions in the flow that you don't formalize into memory notes.

Example shape:

```
{
  "memorySearch": {
    "sources": ["memory", "sessions"]
  },
  "experimental": {
    "sessionMemory": true
  }
}
```

Design trade-off is that you get coverage but you also ingest noise.

If you index sessions, your flush prompt becomes even more important, because it creates a cleaner, lower-noise signal to retrieve first.

Mmemory contract

If you want memory to behave, you need an explicit contract.

Here's mine:

- **Checkpoint:** before compaction, persist decisions/constraints/state.

- **Ground truth:** separate durable memory from transient chat.
- **Retrieval policy:** define when the agent must search.
- **Noise control:** ensure stored notes are concise and typed (decision vs preference vs task).
- **Observability:** log what was stored and what was retrieved.

I'm writing a deep-dive ebook on **Agentic SaaS,** the emerging design patterns that are quietly powering the most innovative startups of 2026.

You can grab the first chapter here: [Agentic SaaS Patterns Winning in 2026](), packed with real-world examples, architectures, and workflows you won't find anywhere else.

Going beyond config: when you need real memory infrastructure

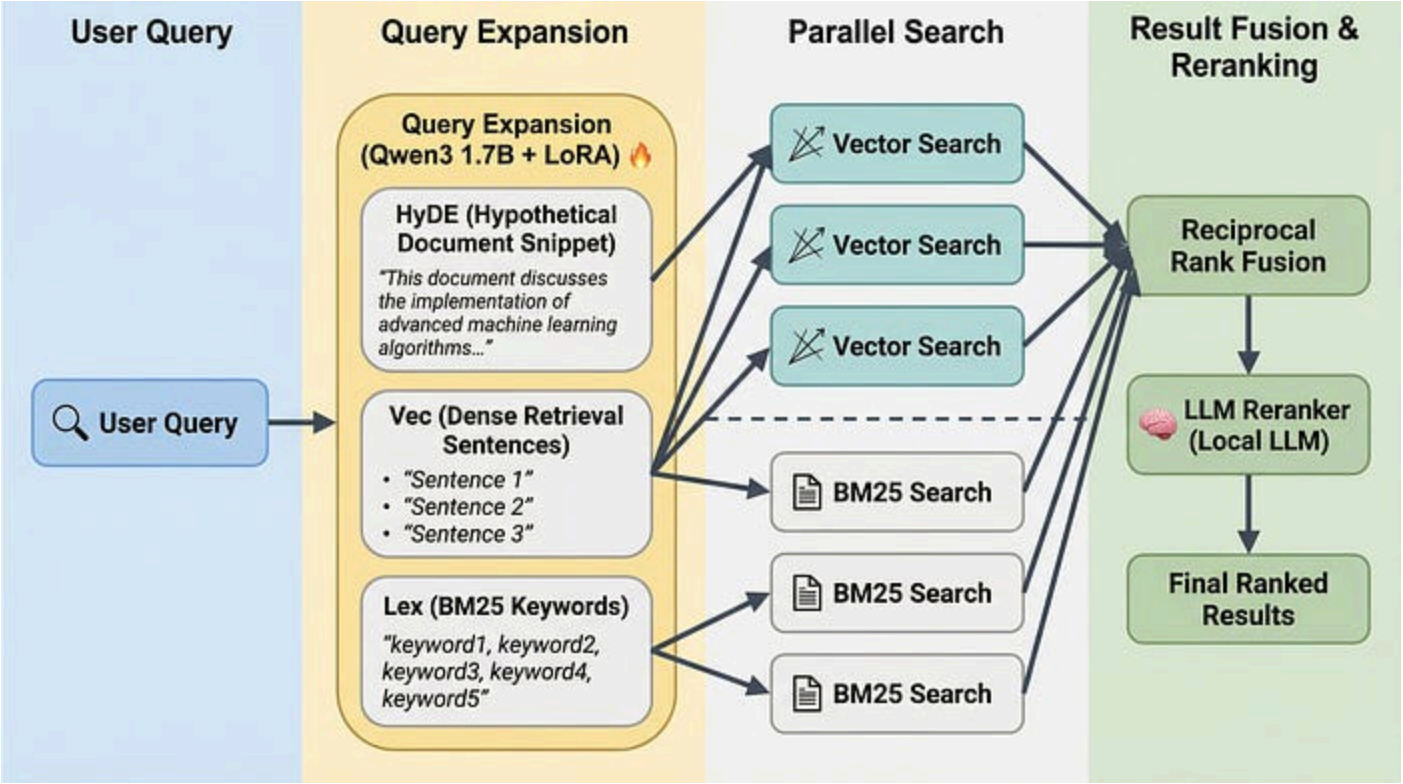
There's a point where **tune the harness** stops being enough.

The signs are predictable:

- you're running multi-day projects
- you're coordinating multiple agents
- your knowledge base is larger than a handful of markdown files
- you care about provenance ("where did this memory come from?")

QMD: treat retrieval as a first-class service

If you set `memory.backend = "qmd"` , you will swap the built-in SQLite indexer for QMD (Query Markup Documents): a local-first search sidecar that combines BM25 + vectors + reranking. Markdown stays the source of truth; OpenClaw shells out to QMD for retrieval.



From an agent architecture perspective, this matters because it makes retrieval:

- stronger
- broader (external docs)
- more configurable

It also introduces a real operational boundary: a sidecar process is a service.

That means you need to think about:

- health checks
- versioning
- index rebuilds
- backup/restore

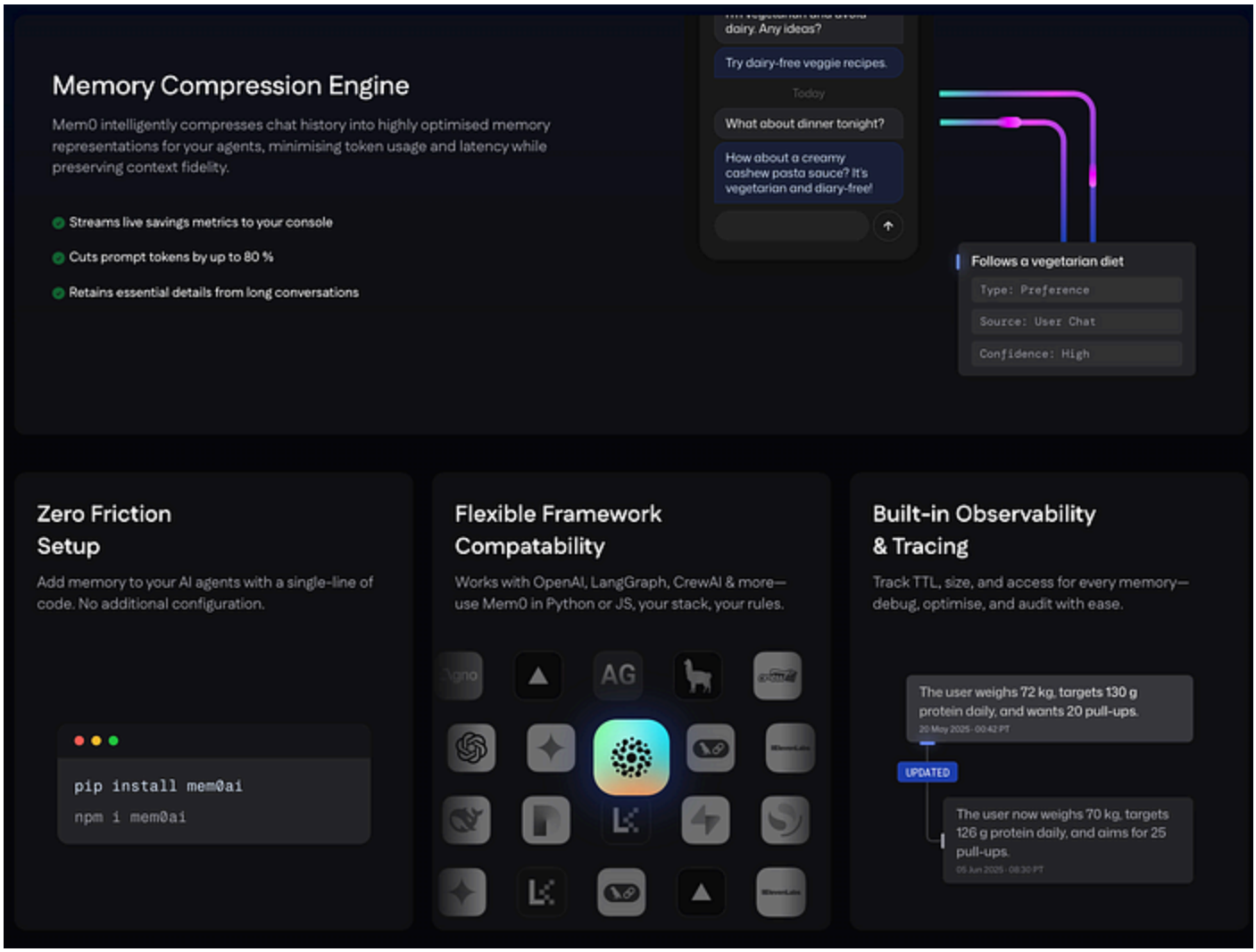
Copy

```
# Install globally (Node or Bun)
npm install -g @tobilu/qmd
# or
bun install -g @tobilu/qmd

# Or run directly
npx @tobilu/qmd ...
bunx @tobilu/qmd ...

# Create collections for your notes, docs, and meeting transcripts
qmd collection add ~/notes --name notes
qmd collection add ~/Documents/meetings --name meetings
qmd collection add ~/work/docs --name docs

# Add context to help with search results, each piece of context will be returned when matching sub documents are r
qmd context add qmd://notes "Personal notes and ideas"
qmd context add qmd://meetings "Meeting transcripts and notes"
qmd context add qmd://docs "Work documentation"

# Generate embeddings for semantic search
qmd embed

# Search across everything
qmd search "project timeline"           # Fast keyword search
qmd vsearch "how to deploy"             # Semantic search
qmd query "quarterly planning process"  # Hybrid + reranking (best quality)

# Get a specific document
qmd get "meetings/2024-01-15.md"

# Get a document by docid (shown in search results)
qmd get "#abc123"

# Get multiple documents by glob pattern
qmd multi-get "journals/2025-05*.md"

# Search within a specific collection
qmd search "API" -c notes

# Export all matches for an agent
qmd search "API" --all --files --min-score 0.3
```

If you're already running multi-agent systems, you'll recognize this as a normal progression: you extracted a component into a service because it was becoming critical.

Mem0: shift memory from "LLM discretion" to "system responsibility"

> Mem0 ("mem-zero") enhances AI assistants and agents with an intelligent memory layer, enabling personalized AI interactions. It remembers user preferences, adapts to individual needs, and continuously learns over time — ideal for customer support chatbots, AI assistants, and autonomous systems.



It changes the write path and compaction resilience.

Mem0 does two important things:

- auto-captures memory without relying on the agent to decide what to save
- auto-recalls relevant memory before the agent responds

Architectural consequence is that you stop trusting the model to be your storage policy.

That is huge because it directly attacks **missed writes** and **compaction loss**, because memory lives outside the context window.

Copy

```
from openai import OpenAI
from mem0 import Memory

openai_client = OpenAI()
memory = Memory()

def chat_with_memories(message: str, user_id: str = "default_user") -> str:
    # Retrieve relevant memories
    relevant_memories = memory.search(query=message, user_id=user_id, limit=3)
    memories_str = "\n".join(f"- {entry['memory']}" for entry in relevant_memories["results"])

    # Generate Assistant response
    system_prompt = f"You are a helpful AI. Answer the question based on query and memories.\nUser Memories:\n{memo
    messages = [{"role": "system", "content": system_prompt}, {"role": "user", "content": message}]
    response = openai_client.chat.completions.create(model="gpt-4.1-nano-2025-04-14", messages=messages)
    assistant_response = response.choices[0].message.content

    # Create new memories from the conversation
```

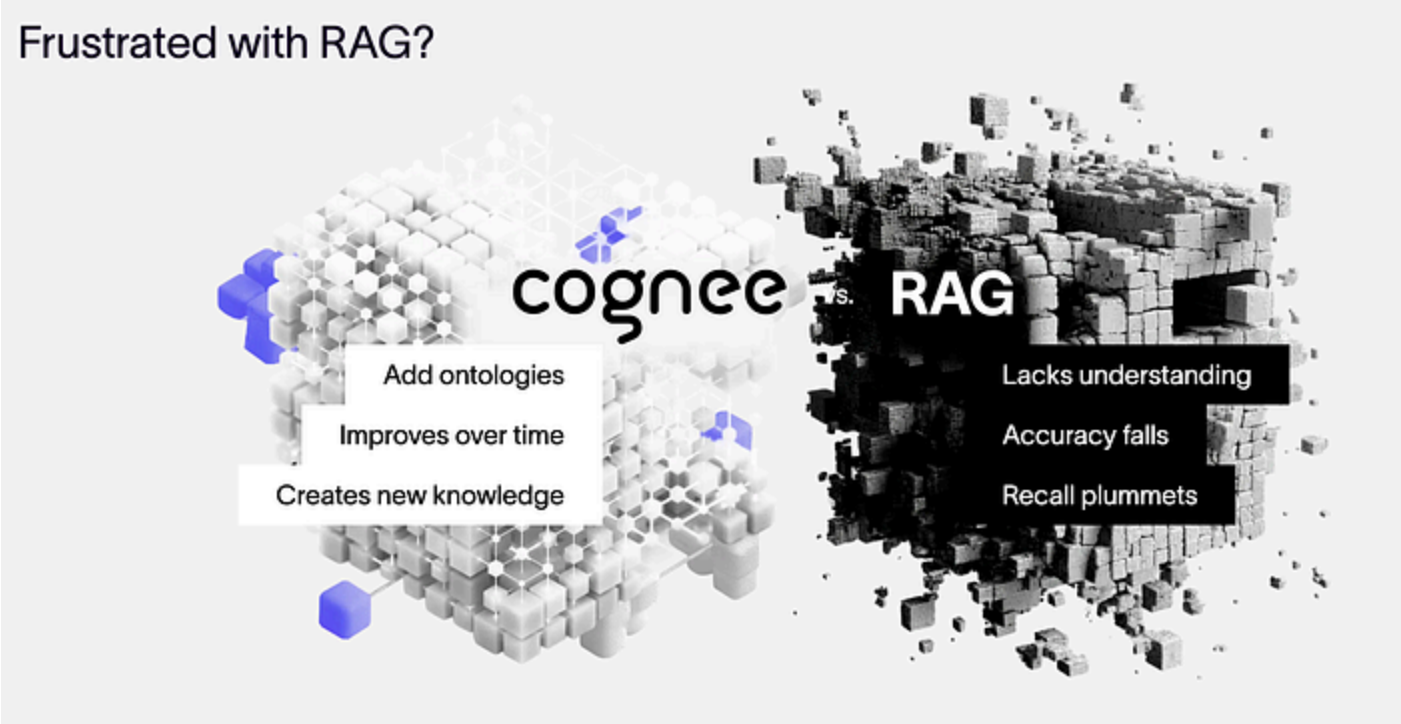
```
    messages.append({"role": "assistant", "content": assistant_response})
    memory.add(messages, user_id=user_id)

    return assistant_response

def main():
    print("Chat with AI (type 'exit' to quit)")
    while True:
        user_input = input("You: ").strip()
        if user_input.lower() == 'exit':
            print("Goodbye!")
            break
        print(f"AI: {chat_with_memories(user_input)}")

if __name__ == "__main__":
    main()
```

Trade-offs you should actually talk about (not hand-wave):

- external dependency (service uptime becomes agent uptime)
- privacy and retention (what are you storing? for how long? how do you delete it?)
- cost per operation at scale

Cognee: when you need structure, not chunks

> Cognee is an open-source knowledge engine that transforms your raw data into persistent and dynamic AI memory for Agents. It combines vector search, graph databases and self-improvement to make your documents both searchable by meaning and connected by relationships as they change and evolve.



It changes the representation.

Vector retrieval is good at similarity but it's bad at reasoning over relationships like ownership, dependencies, and hierarchies unless those relationships are explicitly written in text and retrieved correctly.

Cognee builds a knowledge graph from memory files, e.g. entities and edges.

```python
import cognee
import asyncio
from pprint import pprint


async def main():
    # Add text to cognee
    await cognee.add("Cognee turns documents into AI memory.")

    # Generate the knowledge graph
    await cognee.cognify()

    # Add memory algorithms to the graph
    await cognee.memify()

    # Query the knowledge graph
    results = await cognee.search("What does Cognee do?")

    # Display the results
    for result in results:
        pprint(result)


if __name__ == '__main__':
    asyncio.run(main())
```

That gives you relationship queries, which is not automatically better but it's different.

When graphs are worth it:

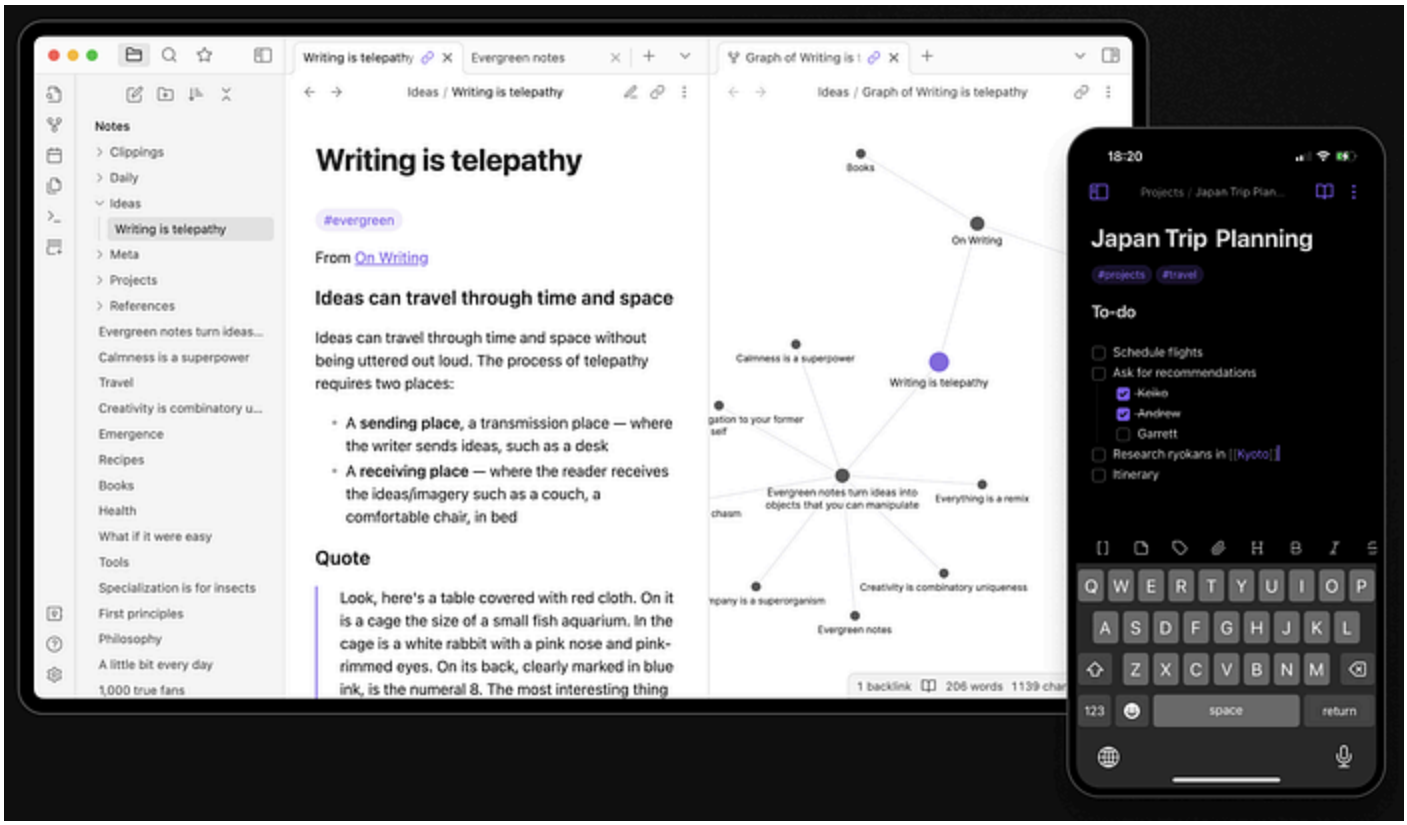
- you need queries like "who owns X" or "what depends on Y"
- you have a multi-agent environment where responsibility and dependencies matter
- you're tired of fuzzy retrieval for structured questions

When graphs become pain:

- your data isn't clean enough to build reliable entities
- you don't have the operational appetite for running the stack
- you're using it because it sounds cool

Obsidian: the human-in-the-loop layer that makes memory less fragile

> Obsidian stores notes privately on your device, so you can access them quickly, even offline. No one else can read them, not even us. With thousands of plugins and themes, you can shape Obsidian to fit your way of thinking. Obsidian uses open file formats, so you're never locked in. You own your data for the long term.



This helps you think about how memory gets curated.

I like Obsidian integrations for a boring reason because they let humans edit what the agent thinks is true.

Your context gives two integration patterns:

- symlink the memory folder into an Obsidian vault so you can review/edit notes
- index the vault via a retrieval backend so the agent can search your curated knowledge

System design perspective this is effectively governance.

You're adding a feedback loop:

- agent writes memory
- human curates it
- retrieval uses the curated store
- agent behavior improves

Agent teams that design memory like an org chart

When you run multiple agents, memory architecture starts to look like organizational design.

You have to come up with a layered approach that maps cleanly to how human teams operate:

- **Per-agent private state:** each agent has its own durable notes and working memory.
- **Shared canonical docs:** a shared directory or repo with team conventions, profiles, and policies.
- **Shared retrieval scope:** all agents can retrieve from the shared canonical docs.

- **Coordinator role:** one agent is responsible for reading canonical docs at start and enforcing consistency.

The key insight is:

- Shared memory without boundaries becomes cross-contamination. Private memory without shared ground truth becomes drift.
- If you've seen two agents disagree on **the correct way we do auth**, that's drift. Fix it the same way you'd fix it in a human org, e.g. write the policy down once and make everyone read it.

The architectural lesson I keep coming back to

Stop expecting memory to be automatic, and in OpenClaw, memory is discretionary unless you make it deliberate:

- deliberate about when you flush
- deliberate about what you store
- deliberate about when you retrieve
- deliberate about how you cope with compaction

If you build agentic systems without these controls, you're building a distributed system where the state disappears randomly and the application sometimes refuses to read from storage.

You would never accept that in a payments service. Don't accept it here just because the interface is chat.

Closing thoughts

If you want agents that do real work, memory is part of the product and here are my practical recommendations:

1. Start with harness-level fixes: flush, pruning, hybrid retrieval, session indexing.
2. Add observability: log what got stored and what got retrieved.
3. If you're still failing, upgrade the memory substrate: QMD for retrieval quality, Mem0 for persistence, Cognee for structured relationships, Obsidian for curation.

Then ask the hard questions your future self will care about:

- What is the **source of truth** when memory conflicts with the current chat?
- How do you test memory behavior across long runs without relying on anecdotes?
- What should be deterministic (policy) versus learned (what to remember)?
- How do you prevent memory from turning into a junk drawer?

If you've shipped an agent system, you already have a scar here. Which one was yours?

Was it missed persistence, missed retrieval, or compaction eating a critical constraint?

Let me know in the comments.

Bonus Articles

Local LLMs That Can Replace Claude Code

Small team of engineers can easily burn >$2K/mo on Anthropic's Claude Code (Sonnet/Opus 4.5). As budgets are tight, you...

medium.com

OpenClaw Variants on $10 Hardware and 10MB RAM

OpenClaw Variants on $10 Hardware and 10MB RAM Most product bugs show up when a simple feature lands on a box with a...

medium.com

Fully Autonomous Companies: OpenClaw Gateway + Routing + Agents

Fully Autonomous Companies: OpenClaw Gateway + Routing + Agents Whether you think it's hype or not, people are already...

medium.com

Codex 5.3 vs. Opus 4.6: One-shot Examples and Comparison

Codex 5.3 vs. Opus 4.6: One-shot Examples and Comparison Just after 9:45 a.m. Pacific on 5 February 2026, Anthropic...

medium.com

Vectorless RAG for Agents: PageIndex Is Why Their Demo Works and Yours Needs "Context"

When I'm building a RAG apps, the thing that quietly kills quality (and budgets) is turning a document into a pile of...

medium.com

Fully Local Agentic Coding on localhost: Claude Code, Codex, llama.cpp, Ollama and Docker MR

This is the setup I wish I had the first time I tried to run Claude Code and Codex-style workflows against local...

medium.com
#openclaw#agent-memory#memo#obsidian#ai-agent

Reporting a Problem

Sometimes we have problems displaying some Medium posts.

If you have a problem that some images aren't loading - try using VPN. Probably you have problem with access to Medium CDN (or fucking Cloudflare's bot detection algorithms are blocking you).

Problem Description Describe your problem here...

Submit Cancel